

ShareKart — Design Document

Services, Contracts & Schema

Team members: [Add your team member names here — one per service, as noted in the Service Design Diagram, services.drawio/pdf]

1. Capability List

From the Assignment 1 brief, the distinct jobs (capabilities) the ShareKart system performs:

- 1 **Register & verify user** — onboard a user and verify identity via Aadhaar authentication.
- 2 **Authenticate / login user** — issue a session to a returning, verified user.
- 3 **List an item** — owner creates a listing (for rent or sale) and the system auto-assigns a price category (<5000 / >5000 / >50,000).
- 4 **Search / browse items** — renters/buyers discover available items by category, price, location.
- 5 **Create rent/buy order** — the central transaction: reserve an item, apply the correct security rules for its price band.
- 6 **Collect security token / deposit** — hold a refundable amount for items priced above ■5,000.
- 7 **Generate & verify bond paper** — additional legal verification/signature for items priced above ■50,000.
- 8 **Capture & compare damage photos** — record before/after photos of the item and raise a damage report if needed.
- 9 **Refund deposit / settle order** — release the held deposit once the item is returned undamaged, or deduct for damage.

(Nine capabilities identified — at least six required. See services.drawio / services.png for the service boundaries these are grouped into.)

3. Service Contracts

Each table lists the operations a service exposes to **other services**. Internals (tables, id formats, SQL) are intentionally hidden — only the operation, its input, output and error cases are exposed.

3.1 User Service (main service — owns Users, Aadhaar Verification, Sessions)

Operation	Input	Output	Errors
registerUser(name, phone, aadhaarNumber, otp)	name: string; phone: string; aadhaarNumber: string; otp: string	userId: string; verificationStatus: enum{PENDING,VERIFIED}	INVALID_AADHAAR, OTP_MISMATCH, DUPLICATE_USER
loginUser(credential, otp)	credential: phone/email; otp: string	sessionToken: string; userId: string	INVALID_CREDENTIALS, ACCOUNT_LOCKED
verifyUser(userId)	userId: string	verificationStatus: enum; kycLevel: enum{BASIC,BOND_ELIGIBLE}	USER_NOT_FOUND
verifyKYC(userId)	userId: string	kycStatus: enum{OK,PENDING,FAILED}	USER_NOT_FOUND, KYC_PENDING

3.2 Item Service (main service — owns Items, Categories, Listings)

Operation	Input	Output	Errors
listItem(ownerId, itemDetails, mode)	ownerId: string; itemDetails: {title, description, price}; mode: enum{RENT,SELL}	itemId: string; category: enum{UNDER_5K, OVER_5K, OVER_50K}	INVALID_ITEM, DUPLICATE_LISTING
checkItem(itemId)	itemId: string	itemDetails: object; priceCategory: enum; availability: bool	ITEM_NOT_FOUND
searchItems(filters)	filters: {category?, mode?, keyword?, location?}	itemList: array<itemSummary>	INVALID_FILTER
getItemPrice(itemId)	itemId: string	price: number; category: enum	ITEM_NOT_FOUND

3.3 Order Service (owns Orders, Bonds — see full spec in section 4)

Operation	Input	Output	Errors
createOrder(userId, itemId, mode, duration)	see full spec, section 4	see full spec, section 4	see full spec, section 4
getOrderStatus(orderId)	orderId: string	status: enum; timeline: array<event>	ORDER_NOT_FOUND
completeOrder(orderId)	orderId: string	finalStatus: enum; depositRefundStatus: enum	ORDER_NOT_FOUND, DAMAGE_PENDING

3.4 Payment Service (owns Payments, Deposits)

Operation	Input	Output	Errors
collectDeposit(orderId, amount)	orderId: string; amount: number	paymentId: string; depositStatus: enum{HELD,FAILED}	PAYMENT_FAILED, INVALID_AMOUNT

3.5 Media Service (owns Photos, Damage Reports)

Operation	Input	Output	Errors
uploadPhotos(orderId, stage, photoFiles)	orderId: string; stage: enum{BEFORE,AFTER}; photoFiles: array<file>	photoSetId: string	INVALID_STAGE, UPLOAD_FAILED

4. Central Operation Specification — createOrder (rent/buy)

Exposed by: **Order Service**. This is the operation every rent or buy transaction goes through.

Inputs

- `userId` (string) — the renter/buyer, already logged in
- `itemId` (string) — the item being rented or bought
- `mode` (enum: RENT | BUY)
- `rentDuration` (object: `startDate`, `endDate`) — required only when `mode` = RENT
- `deliveryAddress` (string)

Outputs

- `orderId` (string)
- `orderStatus` (enum: PENDING_VERIFICATION | AWAITING_DEPOSIT | AWAITING_BOND | CONFIRMED)
- `requiresToken` (bool) — true when item price > ₹5,000
- `requiresBond` (bool) — true when item price > ₹50,000
- `totalAmount` (number) — rent/sale price plus any applicable token amount

Error cases

- `ITEM_NOT_FOUND` — `itemId` does not resolve via Item Service
- `ITEM_UNAVAILABLE` — item already reserved for the requested window
- `USER_NOT_VERIFIED` — caller's Aadhaar verification is not complete (User Service)
- `INVALID_DURATION` — `rentDuration` missing or malformed for `mode` = RENT
- `BOND_REQUIRED_NOT_SIGNED` — item price > ₹50,000 and the bond step has not been completed

Internal details hidden from callers

- The exact price thresholds (5,000 / 50,000) and how they map to `requiresToken` / `requiresBond` are internal business rules, not part of the contract.
- The internal order state machine (PENDING_VERIFICATION → AWAITING_DEPOSIT/AWAITING_BOND → CONFIRMED → IN_PROGRESS → COMPLETED).
- The synchronous/asynchronous calls `createOrder` makes under the hood: `verifyUser()` on User Service, `checkItem()/getItemPrice()` on Item Service, `collectDeposit()` on Payment Service, and `uploadPhotos(stage=BEFORE)` scheduling on Media Service.
- Bond document generation, storage location and the legal template used for items > ₹50,000.
- The internal foreign key linking an order row to a bond row — callers only ever see `requiresBond` and `orderStatus`, never a `bond_id`.

6. Service Validation — the Five Properties

Each service checked against: **reachable**, **self-contained**, **has a contract**, **independent**, **loosely coupled**.

Service	Reachable	Self-contained	Has contract	Independent	Loosely coupled	Notes / Fix
User Service	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	—
Item Service	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	—
Order Service	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✗ No	Currently calls User, Item, Payment and Media services synchronously inside createOrder, which couples its response time to four other services. Fix: make the deposit-collection and before-photo steps asynchronous (event: OrderCreated) so Order Service only calls User/Item Service synchronously for validation, and reacts to DepositCollected / PhotosUploaded events instead of blocking on them.
Payment Service	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	—
Media Service	✓ Yes	✓ Yes	✓ Yes	✓ Yes	✓ Yes	—

Definitions used: *Reachable* — has a network-addressable API. *Self-contained* — owns and can operate on its own data without another service's DB. *Has a contract* — publishes an operation table (input/output/errors) as in section 3. *Independent* — deployable/scalable on its own. *Loosely coupled* — does not block on synchronous calls to many other services for a single request.